

ings is challenging, especially given the high financial and human costs associated with failure. Without a model capable of explaining its decisions, adoption in safety-critical on-orbit missions is unlikely. Additionally, without understanding why these failures occur, it is difficult to implement safeguards. These concerns underscore the need for further study into how, when, and why models fail.

3.2.3 Visualization of Neural Network Predictions

One of the most commonly used methods for CNN classifier visualization is gradient-weighted class activation mapping (Grad-CAM) [123], which measures and visualizes the importance of each pixel of the input image to the CNN’s classification decision. For a given target class and an image, it performs a forward pass with the CNN and extracts the class activation maps (CAMs) of the final convolutional layer. This layer keys in on high-level features used in the CNN’s decision process. Grad-CAM then finds the gradient of the target class score with respect to CAMs from earlier layers using a backward pass. These gradients are pooled spatially across the CAMs. After applying the ReLU activation and upscaling using guided backpropagation [134], the result is overlaid as a heatmap on the input image, where the higher intensities correspond are pixels that were most influential to the class prediction.

Grad-CAM and several other visualization methods (e.g., Grad-CAM++ [16], CNN fixations [99], DeepLIFT [129]) all fall under the class discriminative methods. The performance of these methods heavily rely on the classification being accurate. If objects are classified incorrectly, the visualizations will be distorted or incorrect, defeating the purpose of visualizing the CNNs decision process. To overcome the challenges of the class discriminative methods, Eigen-CAM [7] works based entirely on the forward pass. It assumes that during the training process of the CNN, all non-relevant features learned from the training data will be regularized or smoothed out and only relevant

features are preserved in the model, hence eliminating the dependency on classifier performance and providing class-agnostic analysis of CNN decisions.

Eigen-CAM uses singular value decomposition (SVD) to compute the principal components of the feature maps and projects the input image onto the last convolutional layer. The class activation map is then generated by projecting this output onto the first eigenvector. While Eigen-CAM does not need to compute an expensive backward pass, the SVD can still be computationally intensive.

3.3 Motivation and Methods

For all analysis below, we adopt the YOLOv5 object detector trained to detect solar arrays, satellite bodies, antennas, and thrusters. Details are provided in Section VI.

3.3.1 Patterns in CNN Latent Representations

Inside CNN layers are convolutional filters that, when applied to an input image, compute a feature map. More specifically, the weights inside the filters are learned during model training to extract useful features such as edges and color to create these feature maps, which are useful for the downstream task of object detection. These features are stored as hidden representations of the input data, called feature maps. An example of this process can be seen in Figure 3.2, where we extracted two $6 \times 6 \times 3$ convolutional filters (the two rows of in Figure 3.2: one 6×6 slice applied to each color channel) and the feature maps they produce from the first convolutional module of YOLOv5. Figure 3.2 shows the impact of the filters to achieve specific feature maps.

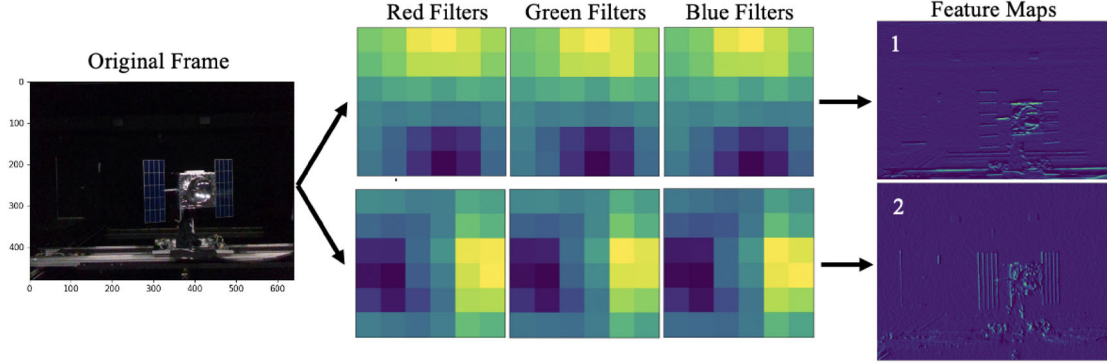


Figure 3.2: Features extracted by convolutional filters of the first layer of YOLOv5

YOLOv5 was trained to perform object detection by minimizing classification and localization errors. During training, these two filters learned to become edge detectors despite no explicit guidance by the optimization process. In particular, the first filter in Figure 3.2 has negative weights (darker) on the bottom of the filter and positive weights (lighter) on the top, both with more intensity near the horizontal center. This combination of blurring (more muted values in the corners) and horizontal dividing line between positives and negatives is similar to the classical top Sobel filter [94] for detecting horizontal lines. Note the horizontal lines on the solar panels are easily visible but the vertical lines are not. The resulting Feature Map 1 clearly enhances horizontal lines. Similarly, the second filter is similar to the classical right Sobel filter for detecting vertical lines, and we see the vertical lines on the solar panel on Feature Map 2.

The collection of feature maps in a convolutional layer contain the latent representations of the data as it flows through the network (as seen above). This is where we will be performing information theoretic analysis because they contain signals with information content that helps us to perform downstream vision tasks, such as the object detection task focused upon in this work.

3.3.2 Entropy and Information Theory

Information theory is the study of information within data signals [24]. Of primary interest to this work are input signals, i.e. images or video feeds, and signals passing through the hidden layers of neural networks. Entropy is an information theoretic measure of uncertainty in the distribution of a random variable X . The (differentiable) entropy of a continuous random variable X valued in $\mathbb{R}^k$ with probability density function (pdf) f is defined as

$$H(X) = \mathbb{E}[-\log f(X)] = - \int_{\mathbb{R}^k} f(x) \log f(x) dx. \quad (3.1)$$

Recent work [96] established the importance of entropy propagation through CNNs to their performance. Hence, we hypothesize the entropy of the internal feature maps of a CNN computed during a forward pass can help explain the model’s decisions.

3.3.3 PEEK

A pixel-level entropy measure applied to the feature maps will be used to analyze the latent data representations within the YOLOv5 architecture and its internal feature maps. This analysis adds transparency to the model’s decision-making process and provides an avenue to detect failure modes and offer remedies for them.

Our method, Probabilistic Explanations for Entropic Knowledge extraction (PEEK), computes a depth-wise entropy-related measure at each “pixel” of the latent feature maps of hidden convolutional layers of CNNs. These latent representations of activations $A \in \mathbb{R}^{l \times w \times d}$ are stacks of feature maps of shape $l \times w$ extracted by the convolutional filters in the layer. We first shift the “pixel” intensities of each feature map to

be positive:

$$A_{ijk}^{\text{pos}} = A_{ijk} + \left| \min_{\{i \leq l, j \leq w\}} A_{ijk} \right| \quad (3.2)$$

Since all feature maps tend to have both positive and negative values, formula (3.2) shifts the values to the positive scale for each feature map separately. This both preserves the relative differences between pixels within a feature map and retains the strength of intense feature maps. PEEK then computes the entropy ($g(x) = -x \ln x$ as defined in a generalized sense in some textbooks [15]) of each depth-wise column of pixels in spatial location (i, j) , resulting in the PEEK intensity at that location:

$$P_{ij} = - \sum_{k=1}^d g(A_{ijk}^{\text{pos}}) \quad (3.3)$$

We apply this PEEK calculation to each spatial location in the hidden representation A to compute a full PEEK map for the selected convolutional layers of the neural network. This PEEK map can be resized and overlaid on the input image, resulting in the highlighting of high-entropy locations in the feature maps corresponding to that location. Figure 3.3 demonstrates the PEEK method. We note PEEK is a general-purpose approach for visualizing CNNs.

We note PEEK can be used on any convolutional layer of a well-trained model for any vision task with any multi-channel 2D imagery. Note we rely on a generalized entropy formula rather than in its typical probabilistic sense, but this is required as it is infeasible to fit a probability distribution to a feature map “pixel” on a sample size equal to the number of convolutional filters in the CNN layer.

In this chapter, we will employ PEEK and Eigen-CAM to visualize the latent data representations within a YOLOv5s model trained for satellite component detection. This will enable a direct comparison of visualization techniques and their effectiveness,

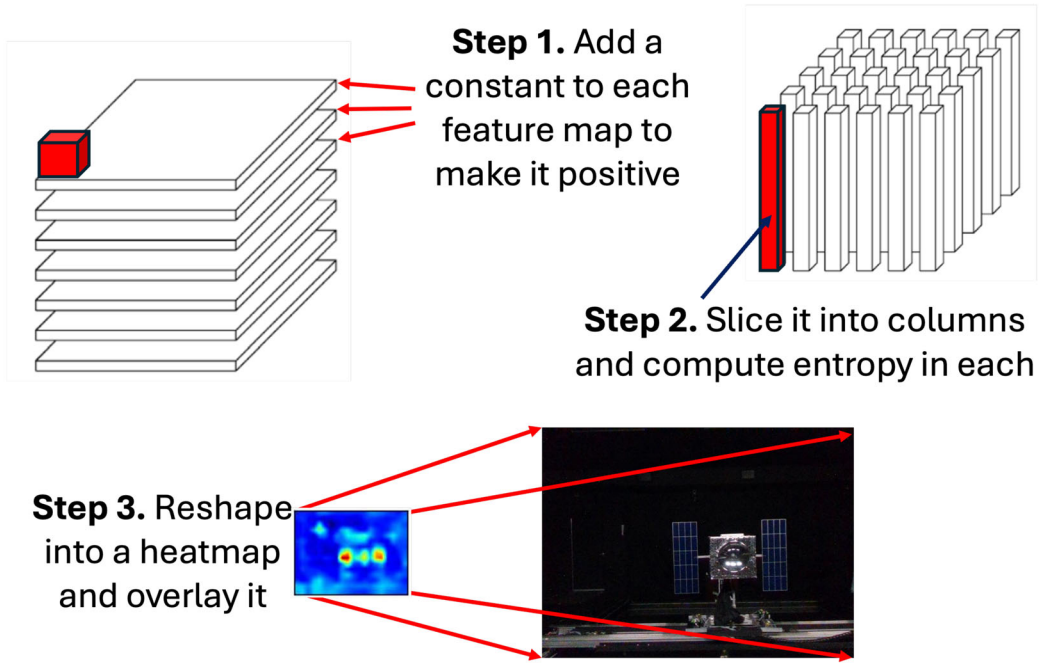


Figure 3.3: PEEK Method Diagram

a study of failure modes with attribution as true risks or artifacts of the experimental environment, and resulting remedies.

3.4 Performance Evaluation and Experimental Conditions

This section specifies the object detection performance metrics, the datasets used for training and testing the algorithms, and the lab setting for hardware-in-the-loop experiments.

3.4.1 Object Detection Metrics

The model's performance is measured in terms of its ability to classify and localize features. This chapter evaluates and compares the mean average precision (mAP) at

the intersection over union (IoU) threshold 0.5. IoU measures the overlap between two bounding boxes by dividing the area of their intersection by the area of their union. It ranges from 0 (no overlap) to 1 (full overlap).

$$IoU = \frac{\text{Area of intersection of bounding boxes}}{\text{Area of union of bounding boxes}} \quad (3.4)$$

A predicted class and bounding box is called a true positive (TP) in object detection if the bounding box has IoU with a ground-truth bounding box above a specified IoU threshold and it is classified correctly. Precision (P) is the ratio of TP detections to all positive detections made by the model, which describes how reliable model predictions are. Recall (R) is the ratio of TP detections to all positive instances in the dataset, which describes how well the model detects objects in images.

$$P = \frac{TP}{\text{Positive object predictions}} \quad (3.5)$$

$$R = \frac{TP}{\text{Ground-truth object instances}} \quad (3.6)$$

A precision-recall (PR) curve is generated by plotting precision versus recall for a range of confidence thresholds required to make detections for each class. Average precision (AP) for a specified class is the area under the PR curve for that class. Mean average precision (mAP) is the mean of class-specific AP s. The resulting mAP is a one-number summary of object detection quality for a specified IoU threshold. We use the standard threshold 0.5, and the metric is called $mAP@0.5$.

3.4.2 Web Satellite Dataset (WSD)

The WSD [92] training data consists of images of spacecraft from Google, mostly generated by STK and Kerbal Space Program as well as Blender-generated images of NASA’s stereo lithography models sourced on the web. A total of 1231 images are collected for the training dataset. Each satellite component (solar array, thruster, antenna, satellite body) is annotated with a bounding box and class label. Figure 3.4 shows the total number of annotations present in the dataset for each component.

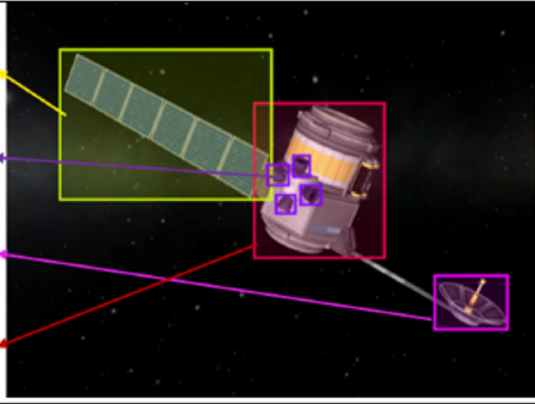
Class	#Annotations	Example
Solar Array	1204	
Thrusters	737	
Antennas	692	
Satellite Bodies	416	

Figure 3.4: Web Satellite Dataset [92]

This dataset was randomly split into training (60%), validation (20%), and testing (20%) subsets. One version of YOLOv5s was trained on the training split using image augmentation to include rotation, black cutout boxes, Gaussian noise, and greyscale. Each augmentation is motivated by issues observed in our experiments. These are shown in Figure 3.5. We refer to this version of YOLOv5 as the “unknown satellite” case since imagery of the real-life satellite mockup discussed below was NOT used in training. This corresponds to use cases where the RSO of interest is unknown in advance.

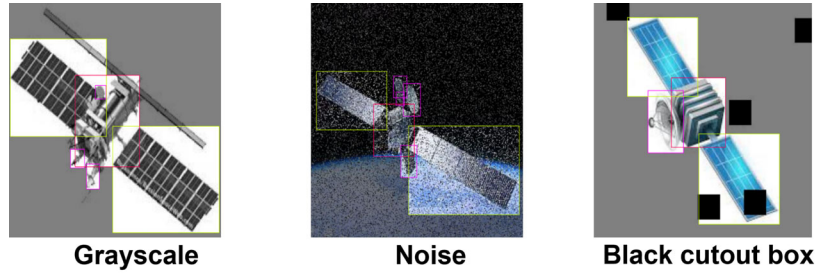


Figure 3.5: Training Dataset Augmentation

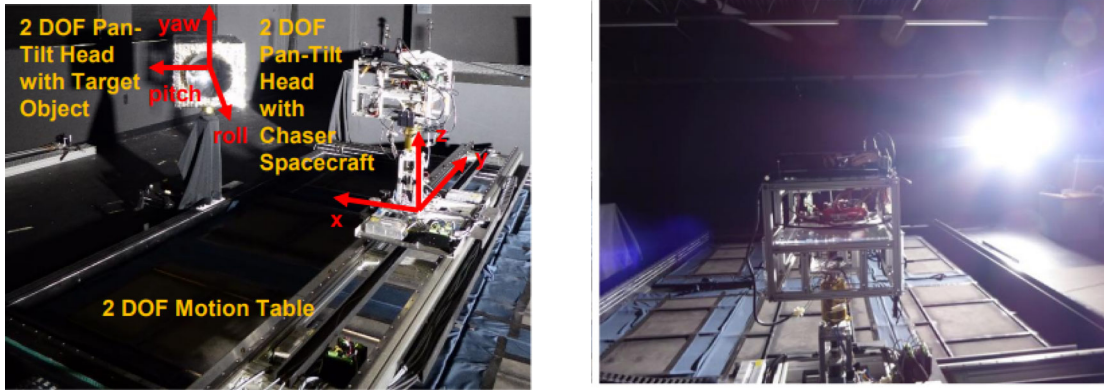
The validation subset was used for tuning the model. The tuned model achieves 0.587 $mAP@0.5$ on the test subset not seen during training. More rigorous testing was then performed in the lab as discussed next.

3.4.3 Testing Dataset and Experimental Setting

This chapter utilizes an experimental testing dataset [6], which includes hardware-in-the-loop image frames extracted at 30 frames per second from two videos of a mock-up satellite, captured using a GoPro 7 camera. In one video, moderate diffuse lighting was used to emulate orbital positions where the sun produces limited shadows on the satellite. In the second, an intense spotlight is used to emulate having the sun in a position such that strong shadows are cast by portions of the satellite onto itself, as depicted in Figure 3.6b. The mock-up satellite yaws about its body axis at a constant angular rate of 2° per second for a full 360° (3 minutes) on the ORION test bed [153] located in the Autonomy Lab at Florida Tech.

The ORION test bed, with its walls, floors, and window panels covered in high-absorptive black paint, as depicted in Figure 3.6, simulates dynamic orbital lighting conditions using a Hilio D12 LED LitePanel, as shown in Figure 3.6b. This setup is designed to evaluate the robustness of the algorithm. In addition to the environmental conditions, the test bed is equipped with a planar maneuver kinematics simulator,

providing a workspace of $5.5\text{m} \times 3.5\text{m}$. The simulator includes two custom-built pan-tilt mechanisms, each with a motion envelope of $\pm 90^\circ$ in elevation and infinite azimuth rotation, enabling attitude changes for both the chaser and the target satellite, as illustrated in Figure 3.6a. The chaser satellite is mounted on a 2-DOF motion platform, simulating relative trajectory motion with respect to the target. The target model replicates the geometry and surfaces typically found on satellites, such as solar panels, parabolic antennas, body panels, and thruster nozzles, as shown in Figure 3.7.



(a) Target satellite mock-up and chaser simulator (b) Testbed simulation of extreme lighting conditions

Figure 3.6: The ORION Testbed in the Autonomy Lab at Florida Tech

Figure 3.7 depicts the dataset of 3600 images of the mock-up via the ORION testbed and annotated with bounding boxes and class labels for its two solar panels, thruster, body, and antenna. The object detectors used below are trained to detect (i.e. localize *and* classify) each visible feature in each frame of the video.

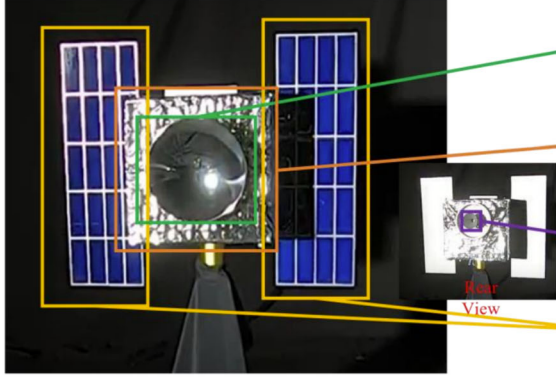
EXAMPLE IMAGE	CLASS	TOTAL ANNOTATIONS
	Antenna	3652
	Body	3692
	Thrusters	758
	Solar Panels	7437

Figure 3.7: Satellite Mock-up Dataset and its Features

Another version of YOLOv5 (small) was trained on frames from the first 80% of this dataset using the same augmentations from Figure 3.5. We refer to this version of YOLOv5 as the “known satellite” case since imagery of the real-life satellite mockup was used in training. This corresponds to use cases where the RSO of interest is known in advance.

The trained YOLOv5 model achieves 0.394 $mAP@0.5$ in the “unknown satellite” case and 0.750 $mAP@0.5$ in the “known satellite” case. The performance is degraded in this real-world setting, but this remains relatively good performance when compared to the object detection literature for hardware-in-the-loop experiments [35, 92].

3.5 Experimental Results

3.5.1 Architecture Analysis via PEEK

The architecture of YOLOv5 as shown in Figure 3.8 is made up of 24 modules separated into the backbone, neck, and head. The backbone (modules 0 to 9) consists of strided convolutional layers, C3 modules (adapted from [150] and optimized), and ends with a

spatial pyramidal pooling “fast” (SPPF) module. The backbone is outlined in Table 3.2 with sub-modules in Figure 3.9.

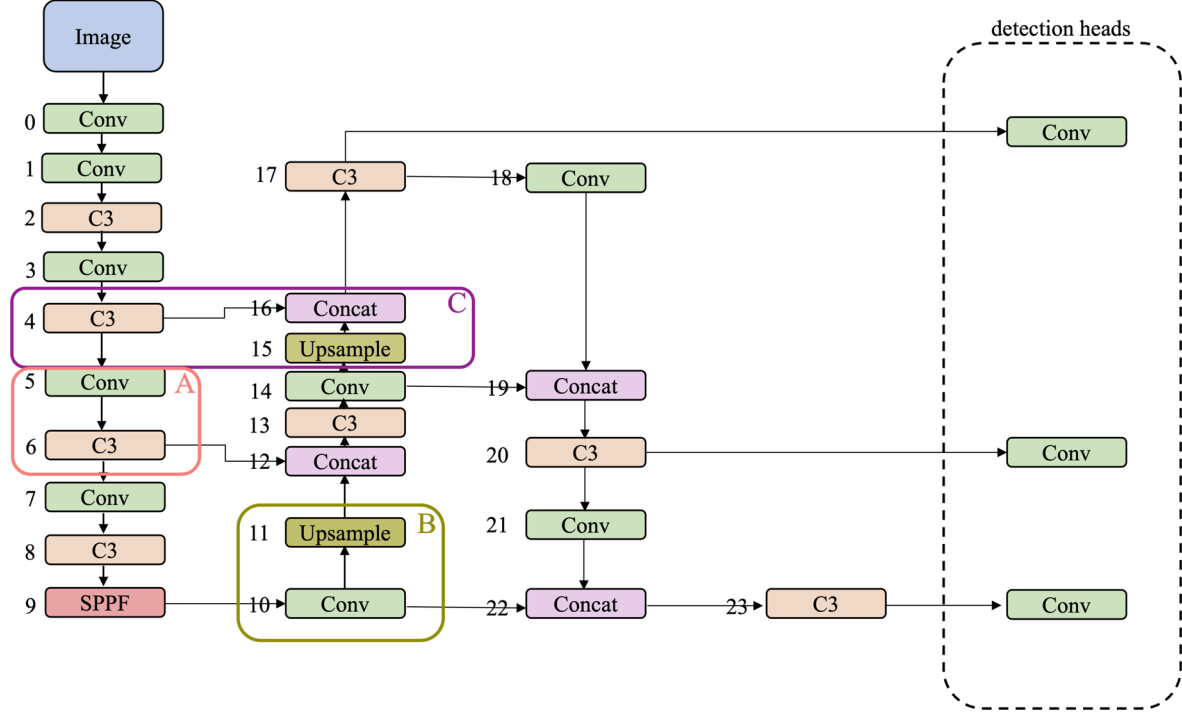


Figure 3.8: YOLOv5 (v6.0/6.1) Architecture

Module	Type	Sub-modules	Hyperparameters (k, s, p, c)	Output Dimension
0	Convolution	Conv + BN + SiLU	(6, 2, 2, 32)	$320 \times 320 \times 32$
1	Convolution	Conv + BN + SiLU	(3, 2, 1, 64)	$160 \times 160 \times 64$
2	C3	w/ 1 Bottleneck	(1, 1, 0, 32 : 32 : 64)	$160 \times 160 \times 64$
3	Convolution	Conv + BN + SiLU	(3, 2, 1, 128)	$80 \times 80 \times 128$
4	C3	w/ 2 Bottlenecks	(1, 1, 0, 64 : 64 : 128)	$80 \times 80 \times 128$
5	Convolution	Conv + BN + SiLU	(3, 2, 1, 256)	$40 \times 40 \times 256$
6	C3	w/ 3 Bottlenecks	(1, 1, 0, 128 : 128 : 256)	$40 \times 40 \times 256$
7	Convolution	Conv + BN + SiLU	(3, 2, 1, 512)	$20 \times 20 \times 512$
8	C3	w/ 1 Bottleneck	(1, 1, 0, 256 : 256 : 512)	$20 \times 20 \times 512$
9	SPPF	(see diagram below)	(1, 1, 0, 256 : 512)	$20 \times 20 \times 512$

Table 3.2: YOLOv5 (v 6.0/6.1) Backbone Modules, including output dimensions from each module assuming 640×640 RGB image inputs. For convolutional layers: k = kernel size, s = stride, p = padding, c = filters in layer.

As the convolutional filter scans over the input, it extracts patterns from image patches spanning the entire input but represents them in a lower dimension in the next layer. This shrinks the feature map shapes by half while still representing the entire image, so each pixel of the output represents a larger image patch. The next layer scans over those output feature maps so they extract patterns from regions that represent more space in the original image. Therefore, this portion of the backbone extracts features at successively larger spatial scales. Last in the backbone is SPPF [48], which performs pooling at different scales, helping the model capture features useful for detecting objects of various sizes.

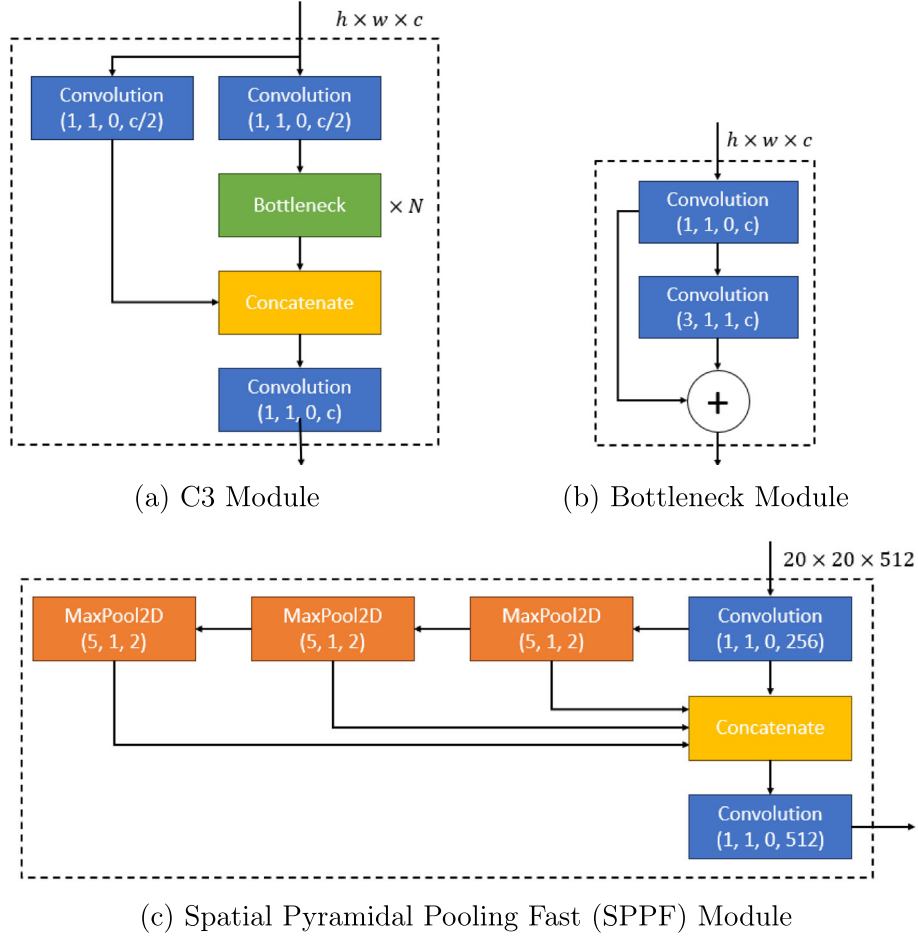


Figure 3.9: YOLOv5 Modules (Convolutions are Conv + BN + SiLU, as above).

The neck (modules 10 to 23) synthesizes features extracted at different scales and with different levels of abstraction. Modules 10-17 work with the backbone to form a feature pyramid network (FPN) [78]. The FPN treats the successively smaller feature maps learned by the backbone as the levels of the pyramid. The bottom of the pyramid corresponds to larger feature maps that have high enough resolution to encode small, fine-grained features. As we move up the pyramid, the model starts to learn larger features. While the FPN focuses on features of the objects themselves, modules 18-23 form a path aggregation network (PANet) [83] that helps estimate object positioning. It also shortens the path from detection heads that focus on detecting small, medium,

and large-scale objects to appropriately-sized features of those objects learned by the FPN.

The heads process features from modules 17, 20, and 23 from the neck and predict bounding boxes and class probabilities for each object identified in the input image. After performing non-max suppression to remove redundant detections, the entire architecture is trained to minimize errors for objects detected by the heads in terms of both object classification and localization by bounding boxes.

PEEK reveals numerous intuitive results regarding the architecture of YOLOv5. In Figure 3.10a, the high-entropy features in Module 5’s feature maps include the satellite and its features as intended, but also the metal tracks along the ground and the door frame on the wall to the left. In addition, the satellite’s solar panels, body, and antenna fade into one another, which would make disentangling the objects difficult. The C3 Module and its bottlenecks reduce the relative entropy of the extraneous features and provide clear separation between the satellite features. Upsampling Module 11 from the neck simply performs nearest neighbor upscaling to increase the resolution of the inputs. As expected, it has no impact on the PEEK features aside from making them higher resolution like the feature maps: as we see in Figure 3.10b, they just become more pixelated. Lastly, PEEK demonstrates that Concatenation Module 16 successfully fuses the well-defined features in Module 4 and more abstract features from Module 10 into intense but disentangled satellite features and well-segmented extraneous features on the back wall of the lab, as seen in Figure 3.10c.

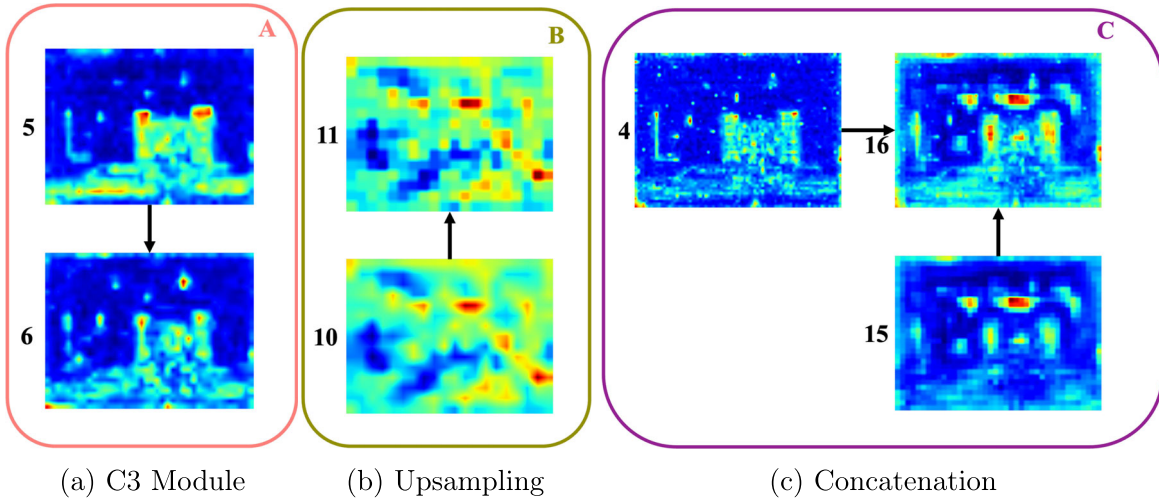


Figure 3.10: The effects of architectural structures on latent representations as revealed by PEEK on the image from Figure 3.2

3.5.2 PEEK vs Eigen-CAM: Visual Explanations and Runtimes

PEEK and Eigen-CAM both extract hidden representations computed in a forward pass through the network and provide visual cues into the model’s detection process. In Figure 3.11a, we see a frame from a synthetic satellite video with YOLOv5’s detections annotated. To analyze how the algorithm made its decisions, we compare Eigen-CAM and PEEK maps of five modules from the model’s architecture in Figure 3.11b. Both methods highlight important features but show key differences in how they capture information.

Eigen-CAM, by using Singular Value Decomposition (SVD), is limited to capturing only the linear relationships in the feature space. It projects the class activation output onto the first principal component which highlights the most prominent linear directions in the data, but will miss the non-linear complexity present in the features [7].

In contrast, PEEK computes the estimated entropy of each pixel through a stack of feature maps in a layer. This estimated entropy evaluates how information is distributed, regardless of whether it arises from linear or non-linear transformations. This

allows PEEK to provide a more comprehensive representation of the learned features, as entropy reflects the overall information content. Since PEEK is agnostic to the underlying structure, it is able of capturing both linear and non-linear information.

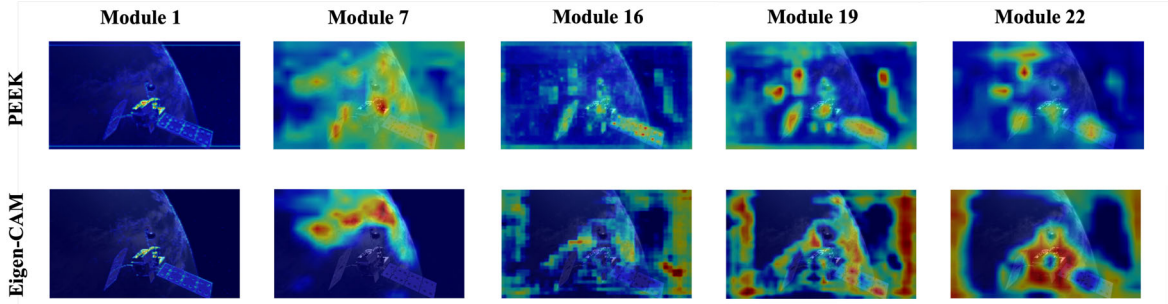
For example, in Module 1, both Eigen-CAM and PEEK effectively highlight edges, particularly on the satellite itself. However, in Module 7, both methods show some distraction by the Earth, though PEEK’s strongest activations remain more concentrated on the satellite, while Eigen-CAM’s focus drifts towards the background, reflecting its limitation in capturing the non-linear dependencies between features for localizations.

As we move deeper into the network, the differences become more apparent. In Modules 16 and 19, PEEK remains focused predominantly on the satellite, though it shows some minor distraction from space and the Earth. Eigen-CAM, however, struggles to maintain focus and is largely drawn towards the boundaries of the image. This pattern is consistent in Module 22, where PEEK is sharply concentrated on the location of the detections, while Eigen-CAM continues to focus on surrounding areas.

These results demonstrate that PEEK provides visual explanations with greater explanatory value for object detection tasks due to its ability to capture both linear and non-linear relationships within the network. By leveraging entropy, PEEK offers a more nuanced and detailed view of the model’s decision-making process, particularly in complex scenes where non-linear features play a crucial role.



(a) YOLOv5 Detections in Synthetic Video Frame



(b) PEEK vs. Eigen-CAM

Figure 3.11: PEEK Analysis of YOLOv5 Satellite Component Detector on a Synthetic Image

Further, the efficient PEEK calculations run far faster than Eigen-CAM. To compare them, we ran both methods 10 times, computing heatmaps for all 23 layers of YOLOv5 for the same input frames, and averaged the runtimes for each layer in Figure 3.12. The compute costs rise and fall with the dimensions of the latent representations of each layer: higher at the top of the backbone in layers and top of the FPN layer 16, and lower elsewhere. Overall, PEEK runtimes are 100 to 1000 times faster than Eigen-CAM while exhibiting qualitative visual benefits demonstrated above. Further, PEEK adds only a single $640 \times 640 \times 1$ array to memory while Eigen-CAM creates this same array along with the added $O(mn + n^2)$ space complexity with its singular value decomposition, which stores its $m \times m$ and $n \times n$ containing the left and right singular

vectors, given a $m \times n$ matrix of activations in the layer. Hence, PEEK is not only 2-3 orders of magnitude faster than Eigen-CAM, but it also has a significantly smaller memory footprint.

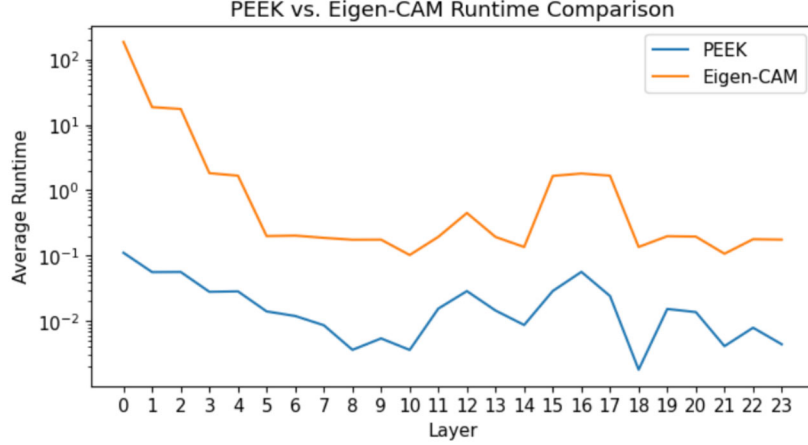


Figure 3.12: PEEK vs. Eigen-CAM Runtimes

3.5.3 PEEK-guided Preprocessing

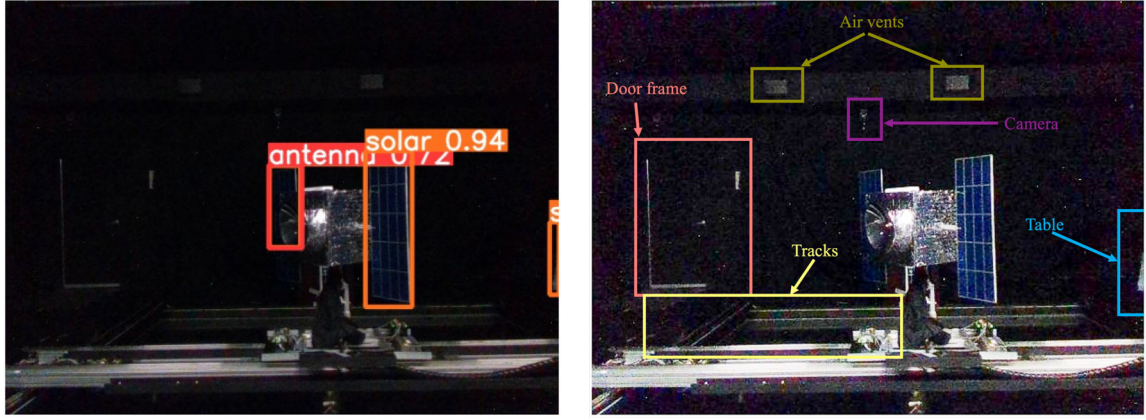
PEEK has allowed us to visualize and better understand the structure of YOLOv5, explain many of its detections, and provide a better visualization of the internal representations of the network.

In Figure 3.13c, we compare PEEK to Eigen-CAM applied to the same detection frame from our lab testing dataset. Eigen-CAM only provides minimal explanation of the detection and errors. For example, in Module 16, Eigen-CAM focuses on the satellite, along with the edges of the frame. This is where we have an incorrect detection of a solar panel on the far right of Figure 3.13a. However, in every other module, it provides us little to no explanation or interpretation.

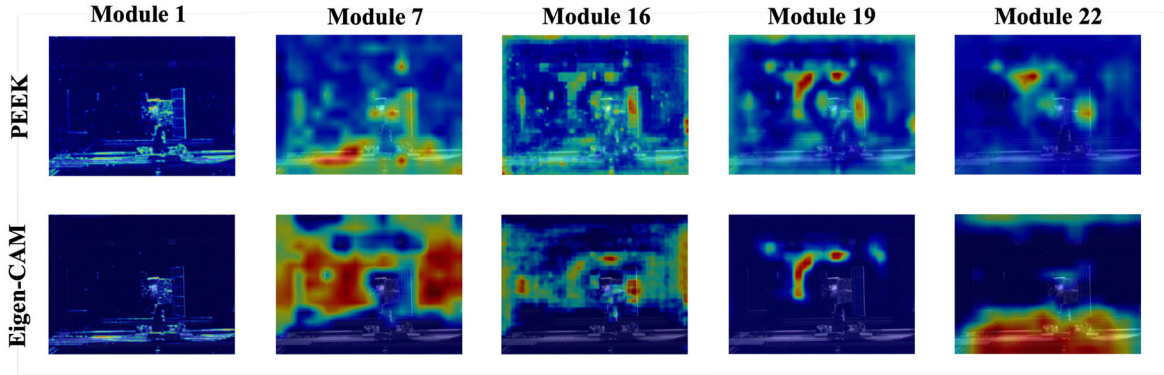
PEEK provides us explanations to possible reasons for errors in the detection along with possible issues with the data itself. In Modules 7, 16, and 19, we have high entropy values seen on the satellite, around the satellite in clear circles or squares, and

a highlighted portion on the far right of the image. The highlighted portion on the satellite was where the algorithm correctly detected and classified a solar panel and antenna. Though the algorithm misses the body of the satellite, in PEEK's modules 19 and 22, we see that the entropy values are weak on areas around the antenna on the body, showing that it is in fact only focusing on the antenna. In Modules 16 and 19, PEEK clearly highlights a shape on the far right that is then detected and classified as a solar panel incorrectly. However, due to the highlighted shapes surrounding the the satellite, it was clear that the algorithm was picking up artifacts in the frames that were not necessarily visible to the human eye without further inspection.

Due to these artifacts seen in PEEK, we decided to take the same frame and brighten the image. Doing this revealed some aspects of our mock set up that could be confusing the algorithm. While the room is prepped to minimize stray light due to unwanted reflections, there are slight exposures that were only noticeable due to PEEK. In Figure 3.13b, we identified the artifacts seen in PEEK in Figure 3.13c as air vents, a camera, a doorframe, and then a table in the far right corner which later was incorrectly detected and classified as a solar panel. This finding shows the value of PEEK and the ability to find artifacts in data, explain errors, and interpret the detections as a whole. This allows us to reevaluate our data and make changes that could improve the quality of our training, minimize biases, and prevent confusion of the algorithm.



(a) Errant YOLOv5 Detections in the Lab (b) Brightened Image, Visible “Distractions”



(c) PEEK vs. Eigen-CAM

Figure 3.13: PEEK for Analysis of YOLOv5 Satellite Component Detector on a Real-World Satellite Mock-Up

Since PEEK has pinpointed several trouble areas in the hardware-in-the-loop dataset, we next explore how PEEK can guide image preprocessing and assess changes in the model’s decisions when we mask input image regions where the PEEK disturbances are associated with false detections. This both builds greater understanding of the decision-making process and guide us to suppress the errors we observe.

Using four distinct approaches, we preprocessed the images to suppress the distracting tracks, air vents, door frame, camera, and table. In all cases, we mask the input images with black pixels using black boxes, black boxes with α -blending into the image, black circles, and subtracting the background outside the extent of the satellite

mockup through the entire dataset. Examples are shown in Figure 3.14.

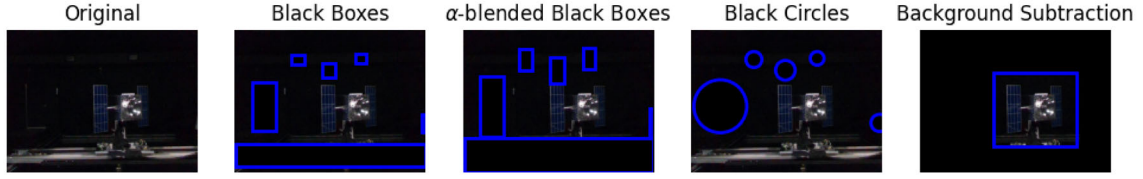


Figure 3.14: Preprocessing Techniques to Counteract PEEK Disturbances (emphasized with blue outlines)

We hypothesized correct detections by the model will be robust to black boxes and background subtraction since the model was trained with blackout box and random crop augmentations. Further, we hypothesize detections are sensitive to shapes as previously suggested by Mahendrakar et al. [90], so blackout circles are expected to trigger false detections of typically-circular features like parabolic antennas and conical thrusters. To test these, we perform YOLOv5 inference in both the known and unknown satellite cases. These hypotheses were tested by both inspecting individual images annotated by their detections with PEEK maps and quantitative object detection metrics.

In Figure 3.15, we see the detections and PEEK maps on a frame of the hardware-in-the-loop test set for each preprocessing technique, using the “known satellite” YOLOv5 model.

The large black boxes placed over the rails at the bottom of the image clearly trigger large PEEK disturbances (but no detections in this frame) while the large box over the door causes a more faint disturbance. The small boxes on top cause no disturbance, presumably because the model was trained to be robust to small (but not large) blackout boxes. Interestingly, black circles consistently trigger antenna detections here. This is expected because the only antenna in the hardware-in-the-loop training split is parabolic and appears circular. Lastly, background subtraction causes PEEK distur-

bances that form a rectangular wave-like artifact as we move away from the satellite. These waves correspond to the receptive fields for representations of the image as it is processed by layers of YOLOv5 that increasingly down-sample it.

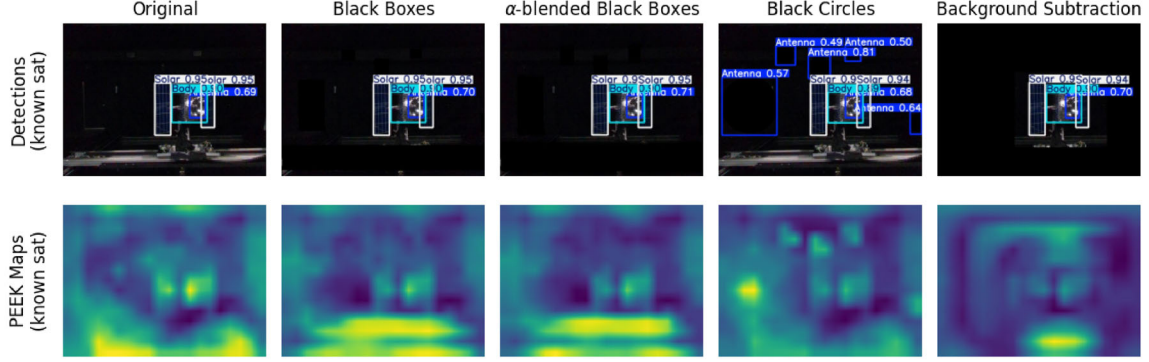


Figure 3.15: Known Satellite Detections and PEEK Maps with Preprocessing

In Figure 3.16, we see the detections and PEEK maps on a frame of the hardware-in-the-loop test set for each preprocessing technique, using the “unknown satellite” YOLOv5 model.

As with the known satellite case, small blackout boxes have no effect, but we see false antenna detections along the edges of the lower blackout boxes covering the rails. These appear like straight lines, which are shaped like the phased-array antennas in the WSD training data. Further, the black circles trigger thruster detections. The intra-class variation in the shape appearance of phased array and parabolic antennas in WSD make the circle prediction less associated with antennas. Thrusters, however, are almost all conical and appear circular in the WSD, so this explains why the black circles are detected as thrusters.

Generally speaking, the preprocessing has less effect on the PEEK maps in the unknown case, although we see the disturbances around the air vents and camera are strongly suppressed by all preprocessing methods. Since this model is trained on a more general task of identifying features of an unknown satellite, small visual variations in the

background seem to disturb the PEEK maps less when masked. We do, however, point out clear disturbances around the thruster detections with black circle preprocessing.

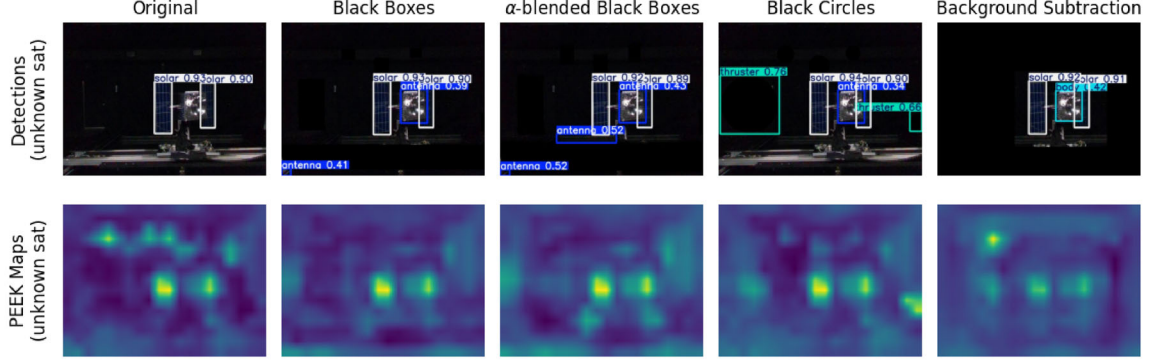


Figure 3.16: Unknown Satellite Detections and PEEK Maps with Preprocessing

We will further note some quantitative differences in the known and unknown satellite models across their entire test sets for different preprocessing techniques. Though the main goal is to explain decision processes, we note background subtraction improves known $mAP@0.5$ by 1.4 percentage points to 0.408 and black box preprocessing improve unknown $mAP@0.5$ by 2.1 percentage points to 0.771.

A notable finding is that the known model performs similarly for antennas under all postprocessing methods *except* black circles. As shown in Table 3.3, all performance measures for antenna sink, especially precision, which from 0.858 (or higher) to only 0.353 with black circle preprocessing. This indicates large amounts of false antenna detections stemming known satellite model’s bias for circular-appearing parabolic antennas, as anticipated. In the unknown satellite case, however, antenna detection performance is not significantly different. With background subtraction, antenna precision is improved 0.180 to 0.338 with no change to recall, which indicates suppressing the background reduces false antenna detections.

Table 3.3: Antenna Metrics for Known Satellite Model with Preprocessing

Precision (P)	Recall (R)	$mAP@0.5$	Preprocessing Method
0.858	0.727	0.804	none
0.887	0.715	0.800	black boxes
0.890	0.728	0.799	α -blended black boxes
0.353	0.650	0.430	black circles
0.914	0.703	0.801	background subtraction

For thrusters, precision and $mAP@0.5$ drop from 0.494 and 0.060, respectively, to below 0.010 with black circle preprocessing in the unknown satellite model, leading to nearly no success finding thrusters. In the known satellite case, performance worsens but not nearly so drastically. As the only class biased in WSD (but not the lab dataset) purely toward circular shapes, this agrees with our hypotheses.

Altogether, we have demonstrated several explanations stemming from PEEK in this subsection. First, PEEK disturbances associated with background noise tend to trigger false detections. Second, masking background regions that cause these disturbances in input images with small black boxes or background subtraction reduces false detections and broadly improves performance. Since blackout boxes and random cropping are used as augmentation techniques, they do not trigger new false detections. Third, masking background regions causing PEEK disturbances with circles causes large amounts of false detections of features that are circular in the respective training datasets. Lastly, such preprocessing techniques are effective only when the model is inherently robust to similar perturbations and does not, therefore, trigger detections.

3.6 Conclusions

In conclusion, the growing congestion of LEO coupled with the impending risks of space debris present a pressing challenge for space agencies and private entities alike. This challenge has been exacerbated by the lack of complete awareness of all orbiting spacecraft, especially non-cooperative ones, making close-proximity operations a high-stakes endeavor. Addressing these issues requires innovative solutions, and one promising approach is the use of autonomous swarms of small chaser satellites.

However, the success of this approach hinges on the deployment of accurate and trustworthy machine vision and sensing systems that can operate in low-compute environments onboard the swarm. While computer vision has made significant strides, the unique constraints of spaceflight hardware and the opaqueness of decision-making in modern vision algorithms have presented substantial hurdles.

This chapter tackled these challenges by introducing the Probabilistic Explanations for Entropic Knowledge extraction (PEEK) method, a novel technique designed to shed light on the decision process of lightweight satellite component detection algorithms. By leveraging information theoretic analysis and internal data representations, PEEK provides valuable insights into the workings of these algorithms, offering explanations in both visual and probabilistic terms. This not only aids in understanding failure modes but also facilitates robust validation and guides the future development of these critical models.

The ability to reevaluate our data in this manner offers a transformative opportunity to enhance the quality of our training datasets and experimental choices. By pinpointing sources of error and bias, we can take proactive steps to rectify these issues, ultimately leading to more accurate and reliable satellite component detection algorithms. Moreover, PEEK empowers us to prevent potential confusion in the algo-

rithm's decision-making process, thereby mitigating risks to the safety and efficiency of operation for autonomous satellite swarms in LEO.

PEEK represents a valuable asset in the ongoing quest to improve the robustness and effectiveness of machine vision systems in space and beyond.

Chapter 4

Balancing Speed, Accuracy, and Efficiency: Exploring Architecture Reduction Through PEEK Variance

4.1 Introduction

Designing optimal neural network architectures that fit the task requirements is often a trial-and-error process, with developers relying on manual adjustments in hopes of balancing performance and efficiency. Solutions like AutoML and Neural Architecture Search [158, 114, 159], and Knowledge Distillation [52] have been introduced to automate this process, however, they are computationally expensive and not practical for every use case. In scenarios that demand edge deployment, where devices operate independently without relying on cloud infrastructure, finding optimal architectures becomes more difficult.

While neural networks have excelled in many tasks such as image classification,

object detection, natural language processing, they are not without limitations. Training these deep networks is computationally intensive, requiring significant time and hardware resources. Deployment on edge hardware like Jetson modules, Raspberry Pi boards, or microcontrollers poses additional challenges, particularly in resource-constrained environments.

Military, aerospace, and medical services often rely on edge AI to perform critical real-time operations. These operations consist of autonomously detecting satellites in space or AI-enabled diagnostic tools on medical devices. These environments demand models that are not only fast and accurate but also small enough to fit within the tight memory and computational limits imposed by these edge devices.

In all these scenarios, quantitative metrics such as Giga Floating Point Operations (GFLOPS) and model size play a critical role. GFLOPS measure the computational cost of running a model, while model size reflects the memory requirements. Lower GFLOPs and small model sizes translate to faster inference times and reduced energy consumption which is essential to operate with limited resources.

Traditional pruning methods, which remove individual weights, help reduce the size and complexity of models but often fail to address the real-world constraints of edge deployment. This chapter explores restructuring pruning, which removes entire layers or modules based on their importance through PEEK, reducing redundancy without compromising performance.

We introduce variance of PEEK as a metric of value and then utilize relative variance contribution percentages to make informed decisions on how to reduce model size, such as where to remove layers and how to reduce complexity. This method not only minimizes accuracy loss but also maximizes efficiency, allowing leaner models with lower GFLOPS and smaller memory. This method provides a systematic framework for building models that are tailored to unique constraints, ensuring that AI technologies

can thrive in environments where computational power and energy resources are at a premium.

4.2 Related Literature

The quest for optimal architectures that balance accuracy and efficiency has been a central pursuit in neural network research. Real et al. [114] introduced Neural Architecture Search (NAS) as an evolutionary approach to discover high-performing network architectures. NAS generates and iteratively evolves multiple architectures based on performance metrics, yielding highly effective designs. However, the computational demands of NAS are immense, often requiring thousands of GPU hours to explore potential architectures, which makes it impractical for resource-constrained environments.

Unlike NAS, which requires extensive computation to exhaustively search for architectures, PEEK pruning provides a layer-level understanding of information flow within the network. By analyzing PEEK maps and variance, developers can identify redundant or low-contributive layers and selectively remove them without extensive search processes. This approach reduces the trial-and-error involved in architectural design and demands far less computational power than NAS.

Other architecture families, such as MobileNet [55] and EfficientNet [140], have been developed specifically for mobile and edge applications. MobileNet achieves efficiency through depthwise separable convolutions, which drastically cut computational costs compared to traditional convolutional layers. While highly efficient, MobileNets are constrained to specific layer types and are often less effective for complex tasks that require high precision or extensive datasets.

EfficientNet optimizes network depth, width, and resolution simultaneously through

compound scaling [140], outperforming architectures like ResNet while maintaining lower computational costs. However, training EfficientNets can still be computationally demanding.

Though both MobileNet and EfficientNet are designed for edge deployment, they are pre-built architectures that may not generalize well across different tasks or hardware constraints. In contrast, PEEK pruning is task-agnostic, using internal activation and variance analysis for the specific task at hand to enable custom pruning tailored to the target application. This results in optimized deployment on specific edge devices and real-world applications like satellite systems, where generalized, one-size-fits-all models may fall short.

Beyond architecture searches, traditional pruning methods such as L1 [45] and L2 [85] pruning focus on weight magnitude but do not consider the semantic significance of a layer’s contribution to the task. PEEK pruning, by visualizing entropy across activations, provides explainable insights into which layers or features are crucial to the model’s decision-making. PEEK maps expose hidden biases and patterns, ensuring the pruned network remains both accurate and reliable [95]). Moreover, approaches like SpaceYOLO [90, 91] have demonstrated the value of variance as a quality measure within networks, underscoring PEEK variance as a strategic, quick and informed choice for pruning.

4.3 Methods

4.3.1 Motivation

In Chapter 3, we introduced PEEK as a way of visualizing how the model makes decisions. We used it as a qualitative analysis tool that guided us to intuitive preprocessing of the data for more accurate predictions. In 4.1, we see an example of a satellite image

being fed into YOLOv5 for object detection. It demonstrates how different modules respond to the same input at different stages of the network. Module 1 produces a PEEK map that highlights fine-grained details of the satellite. Module 19 creates a richer and more comprehensive representation capturing the full areas of interest, although, it can also be falsely triggered by background noise. In Chapter 3, we argue that Module 19 is more informative qualitatively as it focuses on the satellite. However, we lack an approach to evaluate the usefulness of the visualizations.

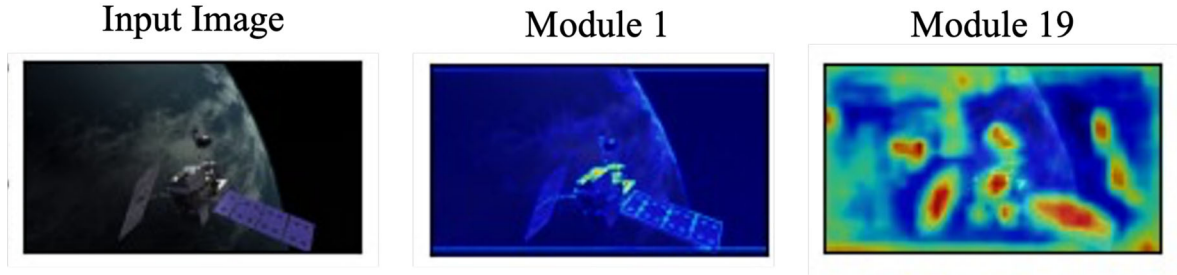


Figure 4.1: A comparison of PEEK for Module 1 and Module 7 in YOLOv5 with satellite data seen in Chapter 3

However, if we were to break down the way we as humans evaluate the PEEK maps in a neural network, we tend to intuitively judge them based on how informative, detailed, and meaningful the representation appears compared to the goal (detecting the satellite components here). In practice, this judgment often corresponds to how much variation is visible in these visualizations. A PEEK map that highlights many distinct patterns or areas of interest across the input data, like the different sections of the satellite, it is likely perceived to be informative. On the other hand, maps with low variations may appear uninformative, suggesting that the layer or module didn't extract meaningful information.

This human intuition aligns closely with the idea of variance of PEEK maps. Variance measures how much individual elements deviate from the mean. If we see areas of high and low entropy distributed meaningfully across the PEEK map, it indicates the

module is capturing a diverse range of patterns, resulting in high variance. Conversely, low variance suggests the module’s entropy is relatively uniform, meaning it is contributing little unique or varied information to the prediction. For example, Module 19 highlights multiple regions, including the objects on the satellite. This suggests that the module has identified diverse and important features, each element in the PEEK map reflects varying levels of activation, resulting in high variance.

In order to explore if we can connect human intuition of the qualitative aspects of the image, we calculate the variance of the PEEK maps.

4.3.2 Variance of PEEK: Mathematical Explanation

The variance measures the spread or dispersion of values in the PEEK map. To compute the variance of PEEK at convolutional layer ℓ , we follow these steps:

First, extract the activations at layer ℓ , $A^\ell \in \mathbb{R}^{l \times w \times c}$. Then, adjust the values to ensure all elements are non-negative by adding the minimum intensity across all channels:

$$A_{ijk}^{\ell, \text{pos}} = A_{ijk}^\ell + \left| \min_{\{i \leq l, j \leq w\}} A_{ijk}^\ell \right|, \quad (4.1)$$

where A_{ijk}^ℓ represents the activation at position (i, j) in the k th channel. Next, we compute the pseudo-entropy for each element using $g(x) = -x \ln x$ and sum depthwise at each position (i, j) across all channels:

$$P_{ij} = - \sum_{k=1}^d g \left(A_{ijk}^{\ell, \text{pos}} \right) \quad (4.2)$$

This produces the PEEK map P^ℓ , a 2D matrix where each element P_{ij}^ℓ represents the activation entropy value (or PEEK) for each spatial location. It is then upsampled

to the size of the input images using bilinear interpolation and antialiasing to produce matrix $Q^\ell \in \mathbb{R}^{L \times W}$ where L and W are the length and width of the input image.

The mean μ_ℓ of the PEEK map is:

$$\mu_\ell = \frac{1}{LW} \sum_{i=1}^L \sum_{j=1}^W Q_{ij}^\ell \quad (4.3)$$

where LW is the total number of elements in the matrix. The variance measures the average squared deviation from the mean:

$$\sigma_\ell = \text{Var}(P^\ell) = \frac{1}{LW} \sum_{i=1}^L \sum_{j=1}^W (Q_{ij}^\ell - \mu_\ell)^2 \quad (4.4)$$

This provides insight into how spread out the values in the PEEK map are. A high variance indicates that certain areas or features have much higher activation than others, suggesting the model is focusing on specific regions. Conversely, low variance suggests a more uniform activation across the map.

- High variance: Significant deviations from the mean suggest that certain activations are much stronger, indicating focused attention on key regions.
- Low variance: More uniform activations suggest generalized attention across the input space, potentially indicating under-optimization or lack of feature specificity.

4.3.3 Understanding the Relationship of Variance in PEEK and Model Accuracy

We trained simple convolutional neural networks (CNNs) on CIFAR-10 dataset to investigate the relationship between variance in PEEK maps and a model’s accuracy.

We started with a small architecture containing three convolutional layers, then incrementally increased the number of layers while maintaining the same optimizers, activations, and layer depths in the prior models until we observed a significant drop in performance. For each architecture, we evaluated the variance of PEEK maps across 100 images to assess how variance correlates with accuracy.

The CIFAR-10 dataset is a standard benchmark dataset for image classification tasks containing 60,000 color images distributed across 10 classes, with each class representing a different category, such as airplanes, cars, birds, and dogs. The images are small (32 x 32 pixels), making this dataset ideal for training and evaluating compact models. CIFAR-10 is challenging due to the classes containing various visual patterns, requiring the models to learn diverse features.

For each architecture, we monitored accuracy on the validation set. When we observed a significant accuracy drop, we concluded that further increasing the number of layers was not beneficial. Each architecture was evaluated independently to track how changes in layer depth impacted performance.

After training, we computed PEEK maps for 100 randomly selected images from the validation set for each network. Variance of PEEK was calculated for each layer across those images to determine how much the entropy values deviated from the mean values. The hypothesis was that layers or networks with higher variance patterns would encode more informative and varied features, potentially leading to better generalizations and higher accuracy.

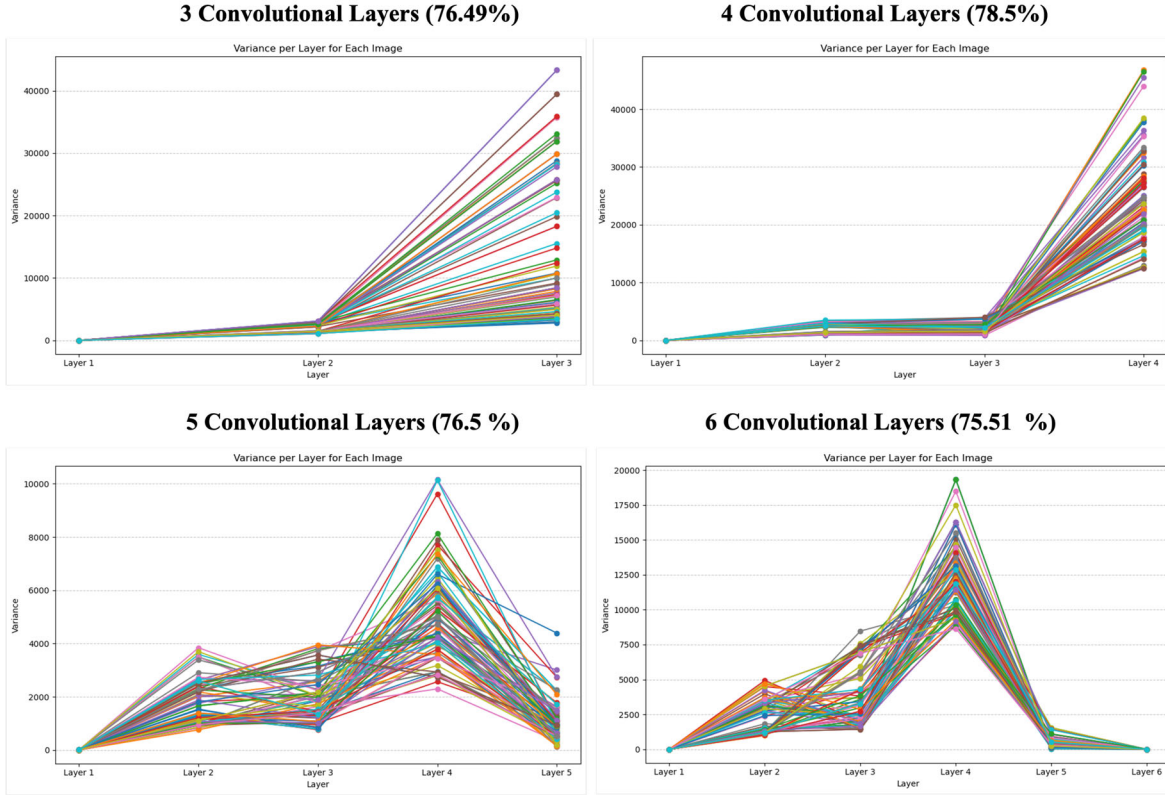


Figure 4.2: This image shows the variance in PEEK maps of 100 images through the layers of 4 simple CNNs trained with incrementally additional layers.

In Figure 4.2 above, it shows the variance trends and accuracy for networks with 3 to 6 convolutional layers. Key findings include:

- **3 Convolutional Layers:** The variance increases sharply at layer 3, suggesting the final convolutional layer plays a significant role in capturing meaningful patterns. However, the overall accuracy of this model is lower compared to deeper models, indicating that the architecture may be too shallow to capture all relevant features.
- **4 Convolutional Layers:** With the inclusion of an additional layer, the variance increases a slightly more gradually across the layers, especially towards the final layer. This model achieves better accuracy, suggesting that the additional layer

helps the model capture more useful representations.

- 5 Convolutional Layers: This model shows a peak and drop in variance at layer 4. The accuracy decreases compared to the 4 layer network, suggesting that the network may be starting to introduce redundancy.
- 6 Convolutional Layers: Despite the additional layers, the overall accuracy drops lower than the 3 convolutional layer model. The variance peaks at intermediate layers, but falls sharply towards later layers, indicating that the deeper network struggles to maintain meaningful activations across all layers.

This experiment showed that networks with higher variance in PEEK maps at critical layers (such as the last convolutional layers) tend to perform better as the layer 4 network had the highest validation accuracy. This suggests that the variance is an indicator of how well the network is capturing useful patterns. Networks with more layers exhibited peaks and drops in variance, indicating that not all layers contribute equally to the final output. This aligns with the need for restructuring-type pruning, where layers with lower variance are pruned to reduce model complexity without sacrificing performance.

4.3.4 Relative Variance Contribution (RVC)

The concept of Relative Variance Contribution (RVC) allows us to quantify the proportion of variance contributed by each layer in a neural network. Given that each layer ℓ produces its own variance σ_ℓ^2 from its respective PEEK map, we can calculate both the total variance across all layers and the proportion contributed by each individual layer.

The **total variance** for all layers is:

$$\text{Total Variance} = \sum_{\ell} \sigma_{\ell}^2 \quad (4.5)$$

where σ_{ℓ}^2 is the variance of the PEEK map for the ℓ th layer.

The **Relative Variance Contribution (RVC)** for each layer ℓ is defined as the percentage of the total variance that the variance of the ℓ th layer contributes. This is given by:

$$\text{RVC}_{\ell} = \left(\frac{\sigma_{\ell}^2}{\text{Total Variance}} \right) \times 100 \quad (4.6)$$

This formulation ensures that the sum of all RVC values across the layers equals 100%, representing the total distribution of variance contributions across the network.

This approach ensures a precise understanding of how much each layer’s PEEK map contributes to the overall variance. These insights can be used for network pruning, optimization, and improving model interpretability by identifying key layers that influence the variance the most.

4.4 Results

4.4.1 PEEK Pruning VGG-16

In this section, we explore how structural pruning was applied to the VGG-17 architecture using insights derived from the RVC analysis. Structural pruning removes entire layers or modules, reducing the model’s complexity while aiming to maintain performance. This approach is particularly beneficial for edge deployments, where the computational resources are limited.